Simple Linux Utility for Resource Management (SLURM)

EDUCADO-MWGaiaDN Training School on Astro-AI and Machine Learning

Bernd Doser

Heidelberg Institute for Theoretical Studies (HITS)

March 5, 2026

Heidelberg Institute for Theoretical Studies HITS

This workshop is available at
https://github.com/BerndDoser/EDUCADO_Slurm_Workshop

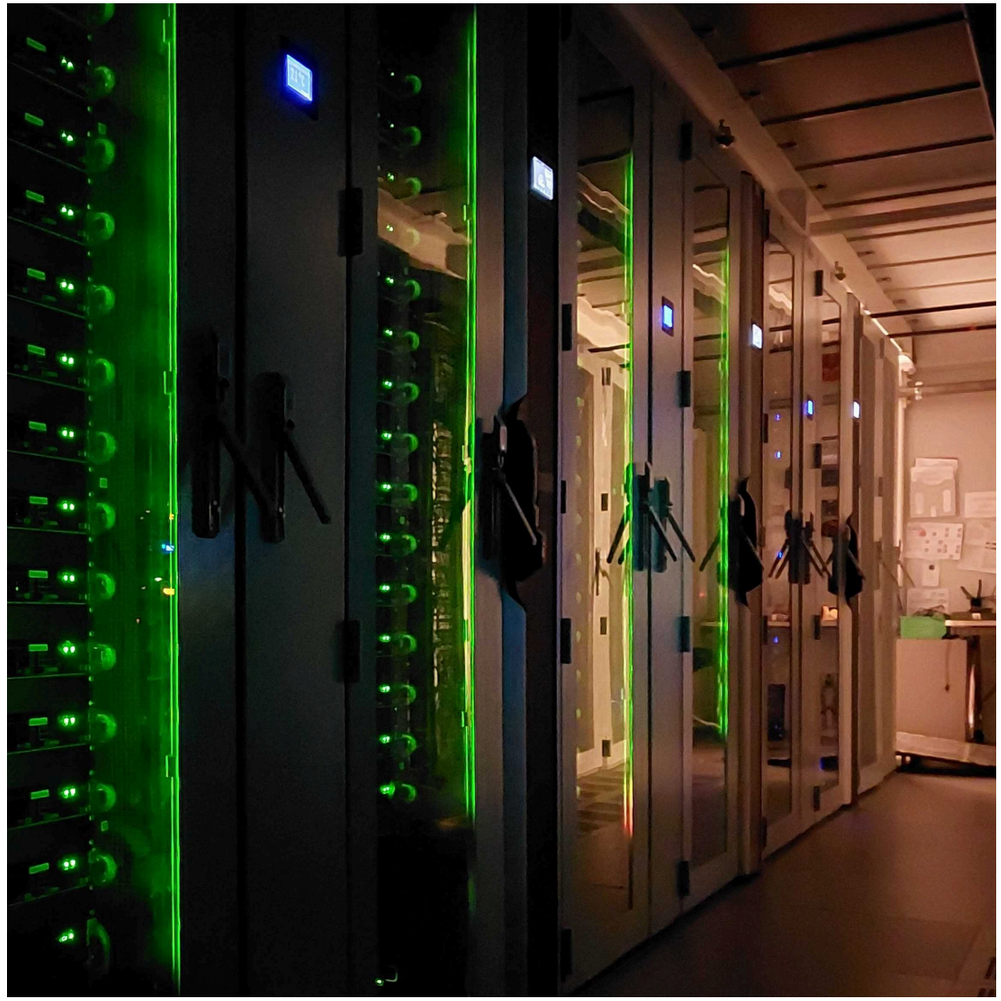
Agenda

- Slurm Basics
  - Understanding HPC Clusters, Partitions and Resources
  - Submitting Jobs and Monitoring Progress
  - Requesting resources (CPUs, memory, time)
- MPI with Slurm
  - Parallel jobs across multiple nodes
- Slurm with StreamFlow
  - Heterogeneous jobs with different resource requirements

Why HPC?

- Hundreds to thousands of CPU/GPU cores
- Fast interconnects (InfiniBand **~400 GB/s**, NVLink **~3 TB/s**, etc.)
- Shared, managed storage (**~9 PB** @ HITS, **417 TB** GPU Direct Storage)
- High-performance software stack (MPI, CUDA, optimized libraries)
- Job scheduling and resource management (**SLURM**, PBS, etc.)
- Access to specialized hardware (FPGAs, TPUs, etc.)

HITS Cascade Cluster (used for hands-on session)



The **Cascade cluster** provides over **3,000 CPU cores**, which can be used for general-purpose parallel workloads and CPU-bound applications.

300 NVIDIA RTX 2080 Super GPUs makes it well-suited for molecular dynamics simulations, machine learning workflows, and large-scale linear algebra computations.

HITS Genoa Cluster

The **Genoa cluster** represents the next generation of high-performance computing. Equipped with state-of-the-art **AMD EPYC Genoa, Bergamo, and Rome processors** as well as the latest **NVIDIA H200 and A100 GPUs**, it provides a versatile platform for both CPU-intensive and GPU-accelerated research workloads.

What is SLURM?

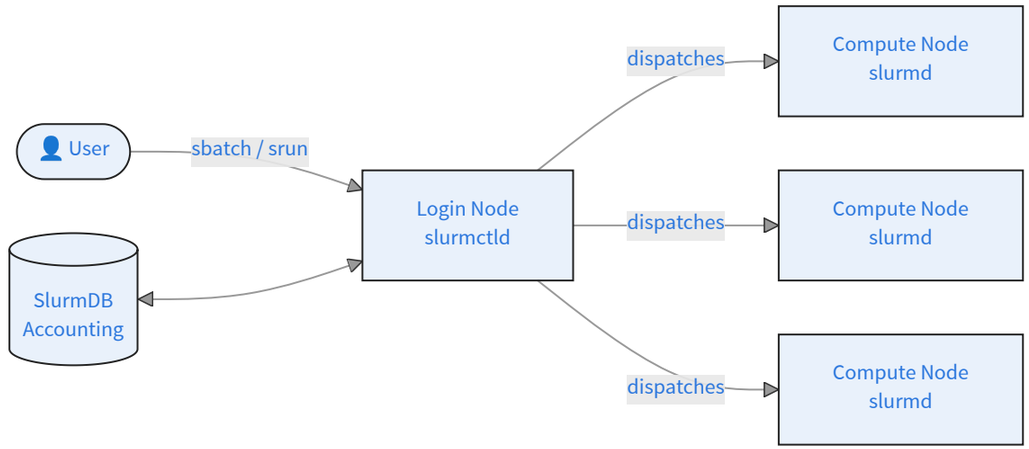
SLURM (**S**imple **L**inux **U**tility for **R**esource **M**anagement) is an open-source workload manager designed for Linux clusters of all sizes.

Yoo, Jette, and Grondona (2003), Jette and Wickberg (2023)

- Keeps track of all nodes, CPUs, GPUs, and memory
- Enforces fair-share policies and quotas
- Accepts job submissions from users (`sbatch`, `srun`, `salloc`)
- Queues jobs and dispatches them when resources are free
- Supports priorities, reservations, and backfill scheduling

SLURM is now part of NVIDIA's HPC software stack (December 2025).

Slurm Architecture



- **Login node** — where users connect via SSH and submit jobs; never run heavy work here
- **Compute nodes** — where jobs actually execute; managed by the `slurmd` daemon
- **SlurmDB** — optional accounting database for job history and fairshare
- **slurmctld** — central controller that manages job scheduling and resource allocation

The Typical SLURM Workflow

1. **SSH** into the login node
2. **Prepare** your script or application
3. **Write a batch script** with `#SBATCH` resource directives
4. **Submit** with `sbatch job.sh` → receive a job ID
5. **Monitor** with `squeue`, `sinfo`, `scontrol show job <id>`
6. **Retrieve results** from output files once the job finishes
7. **Debug** failures with log files and `sacct`

`sinfo`: Cluster status

```
[stud99@cascade-login ~]$ sinfo
PARTITION  AVAIL  TIMELIMIT  NODES  STATE NODELIST
cascade.p*    up 1-00:00:00      1  maint cascade-003
cascade.p*    up 1-00:00:00      3 drain* cascade-[012,028,030]
cascade.p*    up 1-00:00:00      7    mix cascade-[008,027,042,044,054-055,062]
cascade.p*    up 1-00:00:00    111  alloc cascade-[001-002,004-007,009-011,018-026,029,031-039,041
cascade.p*    up 1-00:00:00     26   idle cascade-[013-017,040,043,046-048,052,056,058-061,063-064
debug.p       up    1:00:00      2   idle cascade-[149-150]
karl.p        up 7-00:00:00      1    mix karl
```

The STATE column indicates whether nodes are idle, allocated, in maintenance, or in a mixed state (partially allocated).

`sbatch`: Defining a Job

job.sh

```
#!/bin/bash
#SBATCH --job-name=test                 # Job name
#SBATCH --partition=cascade.p           # Partition to run on
#SBATCH --nodes=1                       # Number of nodes
#SBATCH --ntasks=1                      # Number of tasks per node
#SBATCH --cpus-per-task=4               # Number of CPU cores per task
#SBATCH --mem=8G                        # Memory per node
#SBATCH --time=00:30:00                 # Walltime (hh:mm:ss)
#SBATCH --output=test-%j.out            # Standard output (%j = job ID)

# run your command
echo "Hello from node $(hostname)"
```

- #SBATCH directives to specify job parameters
- Slurm will allocate the appropriate resources and execute the job

`sbatch`: Submit a Job

The batch script can be submitted to the Slurm scheduler using the `sbatch` command:

```
sbatch job.sh
```

which will generate a confirmation message

```
Submitted batch job 18491592
```

indicating that the job was submitted with the `JOBID`, which can be used to monitor or cancel the job.

squeue: Monitor the Job Queue

```
[stud99@cascade-login ~]$ squeue --me
JOBID        PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
18491592   cascade.p  hello_wo  stud99  R       0:11      1 cascade-029
18491601   cascade.p  array_jo  stud99 PD       0:00      1 (Priority)
```

Column	Description
`JOBID`	Unique job identifier
`ST`	State: `R` = Running, `PD` = Pending, `CG` = Completing
`TIME`	Elapsed run time
`NODELIST(REASON)`	Node(s) assigned, or reason if pending

Useful options: `--me` (own jobs), `-u <user>`, `-p <partition>`, `--start` (estimated start time)

`scontrol`: Monitor Job Status

`scontrol` can be used to obtain more detailed information about a job:

```
scontrol show job <job-id>
```

Example output:

```
JobId=18491592 JobName=hello_world
   UserId=stud99(20199) GroupId=aliens(20000) MCS_label=N/A
   Priority=3865470565 Nice=0 Account=(null) QOS=normal
   JobState=RUNNING Reason=None Dependency=(null)
   Requeue=0 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
   RunTime=00:00:11 TimeLimit=00:05:00 TimeMin=N/A
   SubmitTime=2026-02-20T14:51:17 EligibleTime=2026-02-20T14:51:17
   AccrueTime=2026-02-20T14:51:17
   StartTime=2026-02-20T14:51:18 EndTime=2026-02-20T14:56:18 Deadline=N/A
   SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-02-20T14:51:18 Scheduler=Main
   Partition=cascade.p AllocNode:Sid=cascade-login:1286819
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=cascade-029
   BatchHost=cascade-029
   NumNodes=1 NumCPUs=2 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
   ReqTRES=cpu=1,mem=100M,node=1,billing=1
   AllocTRES=cpu=2,node=1,billing=2
   Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
   MinCPUsNode=1 MinMemoryNode=100M MinTmpDiskNode=0
   Features=(null) DelayBoot=00:00:00
   OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
   Command=/home/stud99/EDUCADO_Slurm_Workshop/hands-on/exercise_1/hello.sh
   WorkDir=/home/stud99/EDUCADO_Slurm_Workshop/hands-on/exercise_1
   StdErr=/home/stud99/EDUCADO_Slurm_Workshop/hands-on/exercise_1/hello_18491592.out
   StdIn=/dev/null
   StdOut=/home/stud99/EDUCADO_Slurm_Workshop/hands-on/exercise_1/hello_18491592.out
   Power=
```

`srun`: Interactive Mode

Allocates a node and drops you into an **interactive shell** — useful for debugging, testing, or exploring the environment.

```
srun --partition=cascade.p --time=01:00:00 --pty bash
```

Common options:

Option	Description
`--pty bash`	Open an interactive terminal
`--cpus-per-task=4`	Request multiple CPUs
`--mem=8G`	Request memory
`--gres=gpu:1`	Request a GPU
`--x11`	Enable X11 forwarding

Slurm Tiny Helper

- Cancel a running job: `scancel <jobid>`
- Check past jobs (if available): `sacct -u $USER --starttime=today`
- Email notifications:

```
#SBATCH --mail-type=END  # or ALL
#SBATCH --mail-user=your@email.com
```

- Job dependency: Submit two jobs where the second only runs after the first succeeds:

```
JOB1=$(sbatch --parsable hello.sh)
sbatch --dependency=afterok:$JOB1 hello.sh
```

Fairshare

Fairshare ensures equitable cluster access by **prioritizing users who have used fewer resources** recently.

$$\text{Priority} = w_{fs} \cdot F + w_{age} \cdot A + w_{qos} \cdot Q + \dots$$

- **Fairshare score** $F \in [0, 1]$: closer to 1 means higher priority (less recent usage)
- Score decays over time — past usage matters less as time passes

```
# View your fairshare score
sshare -u $USER

# See job priority breakdown for pending jobs
sprio -l -u $USER
```

Submit jobs when your fairshare score is high to get shorter wait times.

Quality of Service (QoS)

Most clusters use a standard set of QoS levels:

- **debug**: Very high priority, but very short walltime (e.g., 30 mins) and small node limits. Great for testing code before a big run.
- **normal**: The default. Balanced priority and standard limits.
- **long**: Lower priority, but allows jobs to run for 7+ days.
- **scavenger**: Zero priority. Jobs only run if the cluster is empty, and they can be killed (preempted) the moment a "paying" job arrives.

To submit a job with a specific QoS:

```
sbatch --qos=high_priority my_script.sh
```

Why is my job still waiting?

High recent usage lowers your Fairshare score. Use `sprio -l -u $USER` to see where your job ranks among all pending jobs.

Job Arrays

If you have a large number of similar jobs (e.g., parameter sweeps), you can use job arrays to submit them efficiently.

```
#!/bin/bash
#SBATCH --array=0-9
#SBATCH --time=01:00:00
#SBATCH --job-name=array_job

echo "Running task $SLURM_ARRAY_TASK_ID"
```

MPI with Slurm

```
#!/bin/bash
#SBATCH --nodes=2               # Request 2 nodes
#SBATCH --ntasks-per-node=4     # Run 4 tasks per node (total 8 tasks)
#SBATCH --time=1:00:00          # Set a time limit to 1 hour

module load openmpi             # Load your MPI module
srun ./my_mpi_program           # srun handles the launching
```

Use `srun` (instead of `mpirun` or `mpiexec`) to launch parallel tasks across nodes.

It directly communicates with Slurm to launch processes, propagates signals correctly (like cancelling a job), and ensures process binding respects your requests.

GPU Jobs with Slurm

GPUs are managed as **Generic REsources (GRES)** in Slurm.

```
#SBATCH --gres=gpu:a100:2        # 2× NVIDIA A100 per node
#SBATCH --gres=gpu:v100:1        # 1× NVIDIA V100 per node
#SBATCH --gres=gpu:rtx3090:4     # 4× RTX 3090 per node
```

Find available GPU types with

```
sinfo -o "%P %G" | column -t
```

Check actual GPU allocation inside your job

```
echo $CUDA_VISIBLE_DEVICES    # Slurm sets this automatically
nvidia-smi                    # shows only your allocated GPUs
```

Heterogeneous Jobs

Sometimes a single job needs different resources for different parts (e.g., a master process on a CPU node and worker processes on GPU nodes).

Key Features:

- Combine multiple job specifications into one valid job.
- Each component (pack group) has its own resources (nodes, memory, partition).
- Submitted as a single entity with one JOBID.

```
#!/bin/bash
#SBATCH --partition=genoa.p --nodes=2 --ntasks-per-node=32
#SBATCH hetjob
#SBATCH --partition=genoa-deep.p --nodes=1 --gpus=4

srun --het-group=0 ./preprocess
srun --het-group=1 ./train
```

Slurm REST API

- Workflow managers and web portals submitting jobs on behalf of users
- CI/CD pipelines triggering HPC jobs automatically
- Monitoring dashboards polling job and node state
- Python scripts submitting jobs directly

Method	Endpoint	Description
`GET`	`/slurm/v0.0.42/jobs`	List all jobs
`GET`	`/slurm/v0.0.42/job/{job_id}`	Get details of a specific job
`POST`	`/slurm/v0.0.42/job/submit`	Submit a new job
`DELETE`	`/slurm/v0.0.42/job/{job_id}`	Cancel a job
`GET`	`/slurm/v0.0.42/nodes`	List all nodes and their state
`GET`	`/slurm/v0.0.42/partitions`	List partitions

Slurm Reservation

A **reservation** pre-allocates cluster resources (nodes, CPUs, time window) for exclusive use by a specific set of users or accounts.

```
scontrol show reservation  # List all reservations
```

Our reservation of 10 nodes (200 cores):

```
ReservationName=aliens_99 StartTime=2026-03-05T06:00:00 EndTime=2026-03-06T06:00:00 Duration=1-00:00:00
   Nodes=cascade-[001-010] NodeCnt=10 CoreCnt=200 Features=(null) PartitionName=(null) Flags=SPEC_NODES
   TRES=cpu=400
   Users=(null) Groups=aliens Accounts=(null) Licenses=(null) State=INACTIVE BurstBuffer=(null) Watts=n/a
   MaxStartDelay=(null)
```

Users submit jobs into a reservation with:

```
sbatch --reservation=aliens_99 job.sh
```

Slurm with Containers

- Ship complex software stacks in a portable container image
- Enable reproducible research and portable workflows
- Guarantee identical environments across development and production

Apptainer is the **de-facto standard** on HPC clusters.

```
#!/bin/bash
#SBATCH --job-name=container_job
#SBATCH --output=container_%j.out
#SBATCH --time=00:30:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G

module load apptainer

apptainer exec --bind /data:/data my_image.sif python3 /app/run.py
```

StreamFlow

- StreamFlow is a general framework for workflow orchestration
- Relies on the Common Workflow Language (CWL)
  and connects the CWL with a deployment model
- Workflows can be deployed to different environments:
  - Slurm
  - Kubernetes
  - Containers / Apptainer
- Designed for heterogeneous jobs with varying resource requirements
- Example for astronomy data processing pipelines: LOFAR-VLBI Pilo

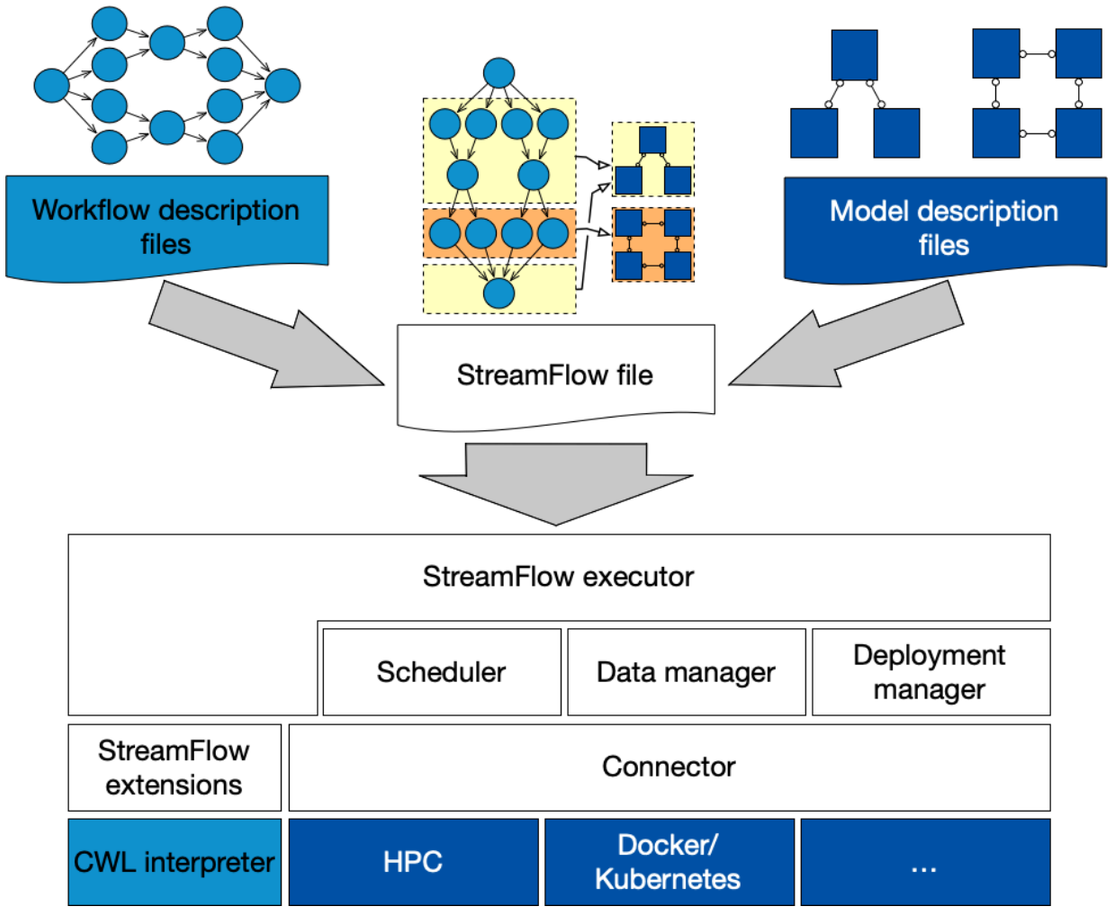
Common Workflow Language (CWL)

workflow.cwl

```yaml
class: Workflow

inputs:
  epochs: int
  train_dataset: Directory
  eval_dataset: Directory
  train_script: File
  eval_script: File

outputs:
  model:
    type: File
    outputSource: train/model
  accuracy:
    type: string
    outputSource: eval/accuracy

steps:
  train:
    run: train.cwl
    in:
      train_script:train_script
```

- Declare the workflow in a YAML file
  - `inputs`
  - `outputs`
  - `steps`
- Inputs and outputs are passed between steps and must be compatible
- Workflows can be shared and reused

StreamFlow Architecture



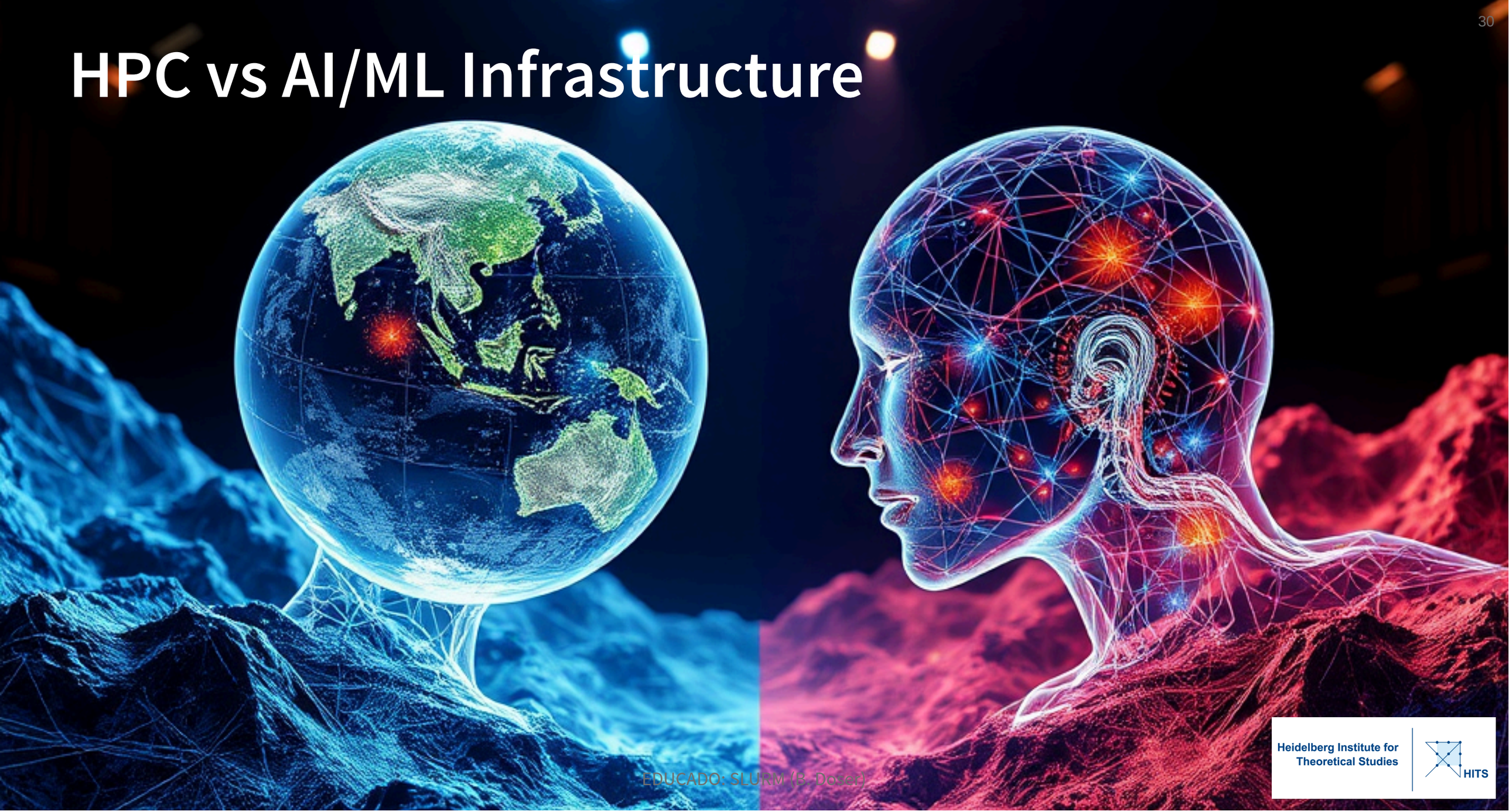
StreamFlow Deployment

StreamFlow connects the CWL workflow to a deployment model

```yaml
workflows:
  workflow1:
    type: cwl
    config:
      file: cwl/workflow.cwl
      settings: cwl/settings.yml
    bindings:
      - step: /train
        target:
          deployment: slurm
          service: genoa-hopper-gpu
      - step: /eval
        target:
          deployment: slurm
          service: cascade
```

```yaml
deployments:
  slurm:
    type: slurm
    config:
      maxConcurrentJobs: 10
      services:
        cascade:
          partition: cascade.p
          nodes: "1"
          mem: 12gb
        genoa-hopper-gpu:
          partition: genoa-hopper.p
          nodes: "1"
          gpus: "h200:2"
          mem: 96gb
```

HPC vs AI/ML Infrastructure



HPC vs AI/ML Infrastructure

HPC workload

- Simulations, modeling, and data analysis
- Large clusters of CPUs and high-speed interconnects
- Parallel computing (MPI)
- Scheduling with Slurm
- Bare-metal compilation
- Distributed file storage

AI/ML workload

- Training and inference of models
- Data-intensive tasks
- Numerous Matrix operations requires accelerators (GPU, TPU, ..)
- Cloud-native deployment
- Kubernetes and containers
- Object storage

Hardware and software requirements differ significantly based on the nature of their workloads and infrastructure.

Summary

- **SLURM** is a widely used job scheduler that manages resource allocation and job scheduling on HPC clusters.
- Understanding the architecture of **SLURM** and how to submit and monitor jobs is essential for effectively utilizing HPC resources.
- **Containers** / **Apptainer** can be used to create reproducible environments for HPC applications.
- **StreamFlow** provides a framework for managing heterogeneous jobs with varying resource requirements.
- HPC and AI/ML workloads have different hardware and software requirements → Slinky

Thank you for your attention!

References

Jette, Morris A., and Tim Wickberg. 2023. “Architecture of the Slurm Workload Manager.” In *Job Scheduling Strategies for Parallel Processing*, edited by Dalibor Klusáček, Julita Corbalán, and Gonzalo P. Rodrigo, 3–23. Cham: Springer Nature Switzerland.

Yoo, Andy B., Morris A. Jette, and Mark Grondona. 2003. “SLURM: Simple Linux Utility for Resource Management.” In *Job Scheduling Strategies for Parallel Processing*, edited by Dror Feitelson, Larry Rudolph, and Uwe Schwiegelshohn, 44–60. Berlin, Heidelberg: Springer Berlin Heidelberg.

Hands-on session

Setup

- Login to the HITS Cascade cluster using the following command:

```
ssh -i <path_to_your_private_key> <username>@cascade-extern.h-its.org
```

The username `studXX` and the private key were provided via email.

- Clone the Workshop Repository and navigate to the hands-on directory:

```
git clone https://github.com/BerndDoser/EDUCADO_Slurm_Workshop.git
cd EDUCADO_Slurm_Workshop/hands-on
```

Exercise 1: Explore the cluster

Run the following commands and answer the questions below:

```
sinfo
squeue
scontrol show partition
```

Questions:

- How many partitions does the cluster have? What are their names?
- How many nodes are in each partition?
- Are any jobs currently running? How can you tell?
- What is the maximum walltime allowed on the default partition?

> Try `sinfo -N -l` for a node-level view and `scontrol show partition <name>` for details on a specific partition.

Exercise 1: Explore the cluster - Solution

- How many partitions does the cluster have? What are their names?

  cascade.p (default), debug.p, karl.p
- How many nodes are in each partition?

  cascade.p: 148 nodes, debug.p: 2 nodes, karl.p: 1 node
- Are any jobs currently running? How can you tell?

  The squeue command shows the jobs in the queue.
- What is the maximum walltime allowed on the default partition?

  cascade.p has a maximum walltime of 24 hours. Default time limit is 2 hours.

Exercise 2: Write your first batch script

Submit and observe

```
sbatch --reservation=aliens_99 hello.sh
```

Now **quickly** (within 30 seconds!) run several times:

```
squeue -u $USER
```

Questions:

- What is your job ID?
- What state is your job in? (`PD` = pending, `R` = running, `CG` = completing)
- On which node is it running?

After the job completes, inspect the output:

```
cat hello_<jobid>.out
```

- Does the hostname match what `squeue` reported?

Exercise 3: Submit a buggy script

Try to submit it:

```
sbatch --reservation=aliens_99 buggy.sh
```

Questions:

- Did the job submit successfully? What error did you get?
- Fix it and resubmit. What happens now?
- After it runs, check both `.out` and `.err` files. What error do you see?
- What is the hidden integer?

Exercise 4: Experiment with resources

Submit the script `resources.sh` and use `scontrol` to inspect:

```
sbatch --reservation=aliens_99 resources.sh
scontrol show job <jobid>
```

Questions:

- Find the `NumCPUs`, `MinMemoryNode`, and `TimeLimit` fields in the `scontrol` output. Do they match what you requested?
- What happens if you request more memory than the node has? Try changing `--mem=9999G` and resubmit. What error do you get?
- Request 1 and 2 GPUs and check the output of `nvidia-smi` in the job. Do you see the expected number of GPUs allocated to your job?

Exercise 4: Resources - Solution

scontrol output with `--mem=512M`:

```
RunTime=00:00:20 TimeLimit=00:02:00 TimeMin=N/A
NumNodes=1 NumCPUs=2 NumTasks=1 CPUs/Task=2 ReqB:S:C:T=0:0:*:*
MinCPUsNode=2 MinMemoryNode=512M MinTmpDiskNode=0
```

–mem=9999G:

```
sbatch: error: Memory specification can not be satisfied
sbatch: error: Batch job submission failed: Requested node configuration is not available
```

nvidia-smi output with `--gres=gpu:1`:

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 565.57.01              Driver Version: 565.57.01      CUDA Version: 12.7     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 2080 ...    On  |   00000000:17:00.0 Off |                  N/A |
| 30%   29C    P8             16W /  250W |       1MiB /   8192MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+
```

Exercise 5: Job Arrays

You need to process 5 “datasets” (we’ll simulate them). Instead of submitting 5 separate jobs, you’ll use a **job array**.

Submit and monitor

```
sbatch --reservation=aliens_99 array_job.sh
squeue -u $USER
```

Questions:

- How many jobs appear in `squeue`? What do the job IDs look like?
- What is the difference between `%A` and `%a` in the output filename?
- After all jobs complete, list the output files. Do you have 5 result files and 5 log files?

Exercise 5: Job Arrays - Solution

What is the difference between %A and %a in the output filename?

- %A is the job array ID, which is the same for all jobs in the array.
- %a is the job array index, which is unique for each job in the array

Exercise 6: MPI Job

You will compile and run a simple **MPI Hello World** program across multiple tasks.

Compile the program

```
cd hands-on/exercise_6
. cascade-gompi-2025b.sh        # Load OpenMPI
mpicc -o mpi_hello mpi_hello.c  # Compile
```

Submit the job

```
sbatch --reservation=aliens_99 mpi_job.sh
```

Questions:

- How many lines appear in the output file? Does each MPI rank print its message?
- Which ranks run on which node? Do all ranks share the same node or are they spread across nodes?
- Change `--ntasks` to `8` and resubmit. How does the output change?

Exercise 6: MPI Job - Solution

Output:

```
Job ID:         18512769
Nodes:          cascade-[127-128]
Total tasks:    4

Hello from rank 0 of 4 on node cascade-127.cluster
Hello from rank 1 of 4 on node cascade-127.cluster
Hello from rank 3 of 4 on node cascade-128.cluster
Hello from rank 2 of 4 on node cascade-128.cluster
```

Exercise 7: Streamflow

Prepare the virtual environment

```
module load Python
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Submit the job

```
streamflow run streamflow.yml
```

Questions:

- Separate the two steps into two different services. What changes do you need to make in the workflow and deployment definitions?

Exercise 7: Streamflow - Solution

```yaml
workflows:
  ...
    bindings:
      - step: /say_hello
        target:
          deployment: slurm
          service: cascade
      - step: /train
        target:
          deployment: slurm
          service: cascade-gpu
```

```yaml
deployments:
  slurm:
    ...
      services:
        cascade:
          partition: cascade.p
          nodes: "1"
          mem: 1gb
        cascade-gpu:
          partition: cascade.p
          nodes: "1"
          gpus: "1"
          mem: 1gb
```